

# Data Engineering Curriculum

Days 33 & 34: Azure Databricks, PySpark, and Data Pipelines

## Day 33: Databricks Fundamentals

---

Azure Databricks is a unified data analytics platform optimized for the Microsoft Azure cloud services. It provides a collaborative environment for data engineers, data scientists, and analysts.

- **Workspace & Notebooks:** The Workspace is an interactive environment where you manage all Databricks assets. Notebooks are web-based interfaces that allow you to run code (Python, SQL, Scala, R) in cells, visualize data, and document the process in real-time.
- **Cluster Management:** Clusters provide the computation resources necessary to run your notebooks and jobs.
  - *All-Purpose Clusters:* Used for interactive analysis and collaborative notebook development.
  - *Job Clusters:* Spun up automatically to run automated jobs, terminating upon completion to save costs.
- **DBFS (Databricks File System):** A distributed file system mounted into a Databricks workspace. It allows you to access data stored in cloud object storage (like Azure Data Lake) as if it were on a local file system.
- **Unity Catalog:** The industry's first unified governance solution for data and AI on the Lakehouse. It provides centralized access control, auditing, and data discovery across multiple workspaces.

## Day 34: Databricks & PySpark

---

PySpark is the Python API for Apache Spark, enabling you to perform large-scale data processing in a distributed manner.

- **Large-scale Transformations:** PySpark DataFrames allow you to clean, aggregate, and transform massive datasets efficiently across cluster nodes.
- **Databricks SQL:** A serverless data warehouse on the Databricks Lakehouse Platform that lets you run native SQL queries with high performance directly on your data lake.
- **Performance Optimization:** Databricks uses the Catalyst Optimizer and the Tungsten execution engine to optimize query execution plans automatically, ensuring operations are compute and memory efficient.

## Step-by-Step: Creating an Azure Databricks Instance

---

1. **Log in to Azure Portal:** Navigate to your Azure Portal.

2. **Search for Databricks:** In the top search bar, type "Azure Databricks" and select the service.
3. **Create a Workspace:** Click **+ Create**.
4. **Configure Basics:**
  - Select your *Subscription* and *Resource Group*.
  - Provide a *Workspace name*.
  - Choose a *Region* close to your data storage (e.g., East US).
  - Select a *Pricing Tier* (Standard is sufficient for basic learning, Premium includes advanced RBAC and Unity Catalog).
5. **Review + Create:** Click Review + Create, wait for validation, then click **Create**.
6. **Launch Workspace:** Once deployment succeeds, navigate to the resource and click **Launch Workspace** to enter the Databricks UI.

## End-to-End Pipeline: ADF to Databricks Data Transformation

**Problem Statement:** Extract a 1M record customer .csv from a source Storage Account using Azure Data Factory, pass execution to Azure Databricks. Inside Databricks, use PySpark to read the CSV, filter for even customerId, store the result into a Delta Lake table, and write it back to the destination storage account in both CSV and Parquet formats.

### Part 1: Orchestration via Azure Data Factory (ADF)

1. **Create Linked Services:** In ADF, create a Linked Service for your source Azure Data Lake Storage (ADLS Gen2) and your target Azure Databricks Workspace (using a Databricks Personal Access Token).
2. **Build the Pipeline:**
  - Create a new pipeline in ADF.
  - Drag a **Databricks Notebook Activity** onto the canvas.
  - Under the "Azure Databricks" tab, select your Databricks linked service.
  - Under the "Settings" tab, browse to the path of your PySpark Notebook in your Databricks workspace.
3. **Trigger Execution:** When the ADF pipeline runs, it acts as the orchestrator. It will spin up the Databricks cluster and automatically trigger the Notebook containing the PySpark logic below.

### Part 2: PySpark Logic in Databricks

Below is the complete PySpark code to be written inside your Databricks Notebook to handle the file reads, filtering, and multi-format writes to ADLS.

```
from pyspark.sql.functions import col

# 1. Define ADLS Paths (ABFSS protocol for ADLS Gen2)
source_csv_path = "abfss://@.dfs.core.windows.net/customer.csv"

dest_delta_path = "abfss://@.dfs.core.windows.net/delta/customers"
dest_csv_path = "abfss://@.dfs.core.windows.net/csv/customers"
dest_parquet_path = "abfss://@.dfs.core.windows.net/parquet/customers"

# Note: Ensure the Databricks cluster has access to the ADLS storage account
# using Service Principal or Access Key configurations.

# 2. Read the source CSV (1 Million Records)
df_customers = spark.read \
    .format("csv") \
    .option("header", "true") \
    .option("inferSchema", "true") \
    .load(source_csv_path)

# 3. Apply Filter: Keep only customers where customerId is an even number
df_filtered = df_customers.filter(col("customerId") % 2 == 0)

# 4. Save to Delta Lake (Lakehouse native format)
df_filtered.write \
    .format("delta") \
    .mode("overwrite") \
    .save(dest_delta_path)

# 5. Save to Destination Storage Account as CSV
df_filtered.write \
    .format("csv") \
    .option("header", "true") \
    .mode("overwrite") \
    .save(dest_csv_path)

# 6. Save to Destination Storage Account as Parquet
df_filtered.write \
    .format("parquet") \
    .mode("overwrite") \
    .save(dest_parquet_path)

print("ETL execution completed and saved to Delta, CSV, and Parquet formats!")
```